



REPORT ON THE OBJECT-ORIENTED PROGRAMMING

School of Information Technology and Engineering

Discipline: Object – Oriented Programming

Project: University system

Professor: Izbassar Assylzhan

Team members:

Assetov Rauan

Yeraliyeva Anel

Aliaidar Abilmansur

The main goal of the project

The main goal of the project is to build a fully functional **University Management System in Java** that supports the complete academic lifecycle in a single integrated platform.

The project was also extended with a REST API layer and a React web frontend, demonstrating the power of a well-decoupled design that can serve both a CLI and a web app without modifying any business logic.

Project description:

1. Design system architecture using **Use Case** and **UML Class Diagrams** based on Clean Architecture (Onion Model).
2. Implement a **Java 21 University Management System**, covering 8 user roles, course registration, grading, research, library, messaging, and IT support.
3. Extend the system with a **REST API** and **React web frontend** on top of the same domain logic.
4. Report, documentation, presentation.

1. Main functionality

1. **Authentication.** Every user enters the system with a username and password. The LoginScreen class drives credential verification, and after a successful login the system routes the user to a role-specific menu.
2. **Admin.** Creates and deletes users of all roles (Student, Teacher, Dean, Manager, Librarian, TechSupport, EmployeeResearcher). Manages personal data, views audit logs, and generates academic reports.

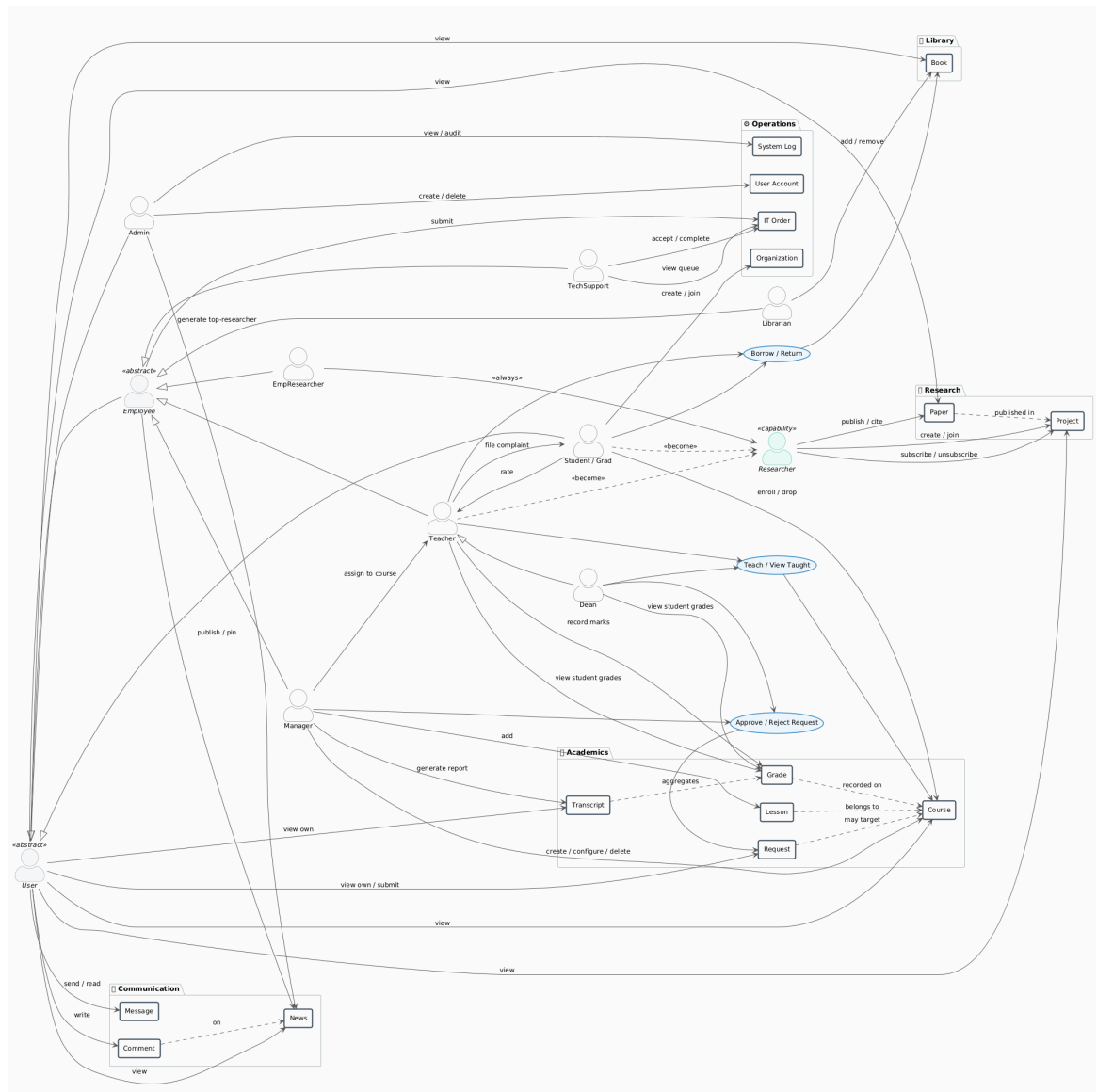
3. **Student.** Registers for and drops disciplines; the system enforces six enrollment rules (credit limit, capacity, prerequisites, schedule conflicts, fail count). Views a personal transcript with computed GPA. Can borrow and return books from the library, submit service requests, join student organizations, and participate in research projects.
4. **Teacher.** Records marks (first attestation, second attestation, final exam). Views enrolled students and the course schedule. Teaches office-hours lessons. Can publish research papers and supervise graduate students (H-index ≥ 3 required).
5. **Manager.** Creates courses, sets capacity and prerequisites, assigns teachers, and schedules lessons via the room scheduler. Views academic statistics and processes study-related requests.
6. **Dean.** Extends Teacher functionality. Receives escalated requests from Manager and approves or rejects them.
7. **Librarian.** Manages the book inventory (add / remove books) and processes borrow/return requests.
8. **Tech Support.** Accepts, processes, and completes IT support orders with status transitions (SUBMITTED $\rightarrow$ ACCEPTED $\rightarrow$ IN_PROGRESS $\rightarrow$ COMPLETED).
9. **Researcher capability.** Any Teacher, Student, or EmployeeResearcher can activate the researcher profile. Researchers can publish papers, join projects, subscribe to journals, calculate their H-index, and view citation rankings.
10. **Messaging & News.** All users can send messages (sorted by urgency level), read inbox, and view/comment on news. Employees can publish news; admins can pin important items.

2. Diagrams

We have created diagrams using StarUML. The UML divides into two parts:

1. Use-case diagram
2. Class diagram

2.1 Use-case diagram:



The use cases are grouped into 5 component areas:

1. Operations
2. Library
3. Academics
4. Communication
5. Research

2.1.1 Operations

- **Admin:** Audit System Log, Create User Account, Delete User Account, Submit IT Order
- **Tech Support:** View IT Order Queue, Accept IT Order, Complete IT Order, Create Organization, Join Organization
- **Employee:** Submit IT Order

2.1.2 Library

- **Admin:** View Book
- **Librarian:** Add Book, Remove Book
- **Student/Grad:** Borrow Book, Return Book
- **User:** View Book

2.1.3 Academics

- **Student/Grad:** Enroll in Course, Drop Course, View Own Transcript, Submit Request, View Own Request, File Complaint, Rate Teacher
- **Teacher:** Teach Course, View Taught Courses, Record Marks (Grade)
- **Dean:** View Student Grades, Approve Request, Reject Request
- **Manager:** Add Grade, Generate Transcript Report, Create Request, Configure Request, Delete Request, View Student Grades, Approve Request, Reject Request, Assign Teacher to Course
- **User:** View Course

2.1.4 Communication

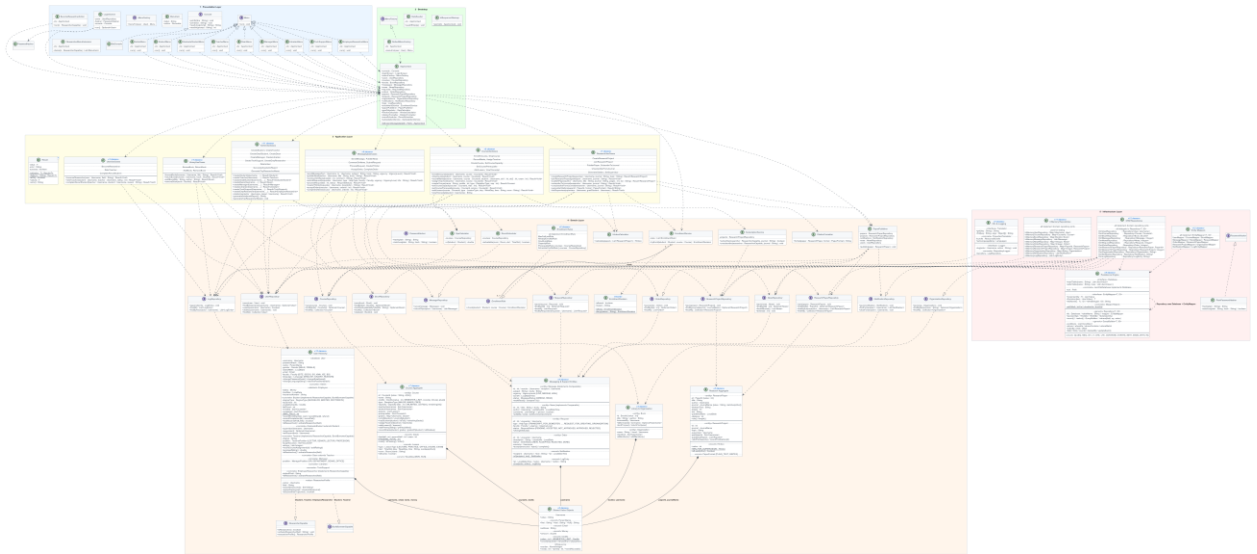
- **User:** Send Message, Read Message, Write Comment, View Comment, View News
- **Admin:** Publish News, Pin News

2.1.5 Research

- **Researcher:** Publish Paper, Cite Paper, Create Project, Join Project, Subscribe to Project, Unsubscribe from Project
- **EmpResearcher:** acts as Researcher
- **Student/Grad:** «become» Researcher

- **Teacher:** «become» Researcher
- **Admin:** Generate Top-Researcher Report

2.2 Class Diagram



3. Code

3.1 Project Structure

```
src/

├─ domain/           Pure business logic, no I/O
├─ application/      Use case orchestration, 46 use cases
├─ infrastructure/   Persistence, auth, i18n
├─ presentation/     CLI menus + REST API, 36 endpoints
├─ bootstrap/        DI wiring: ApplicationContext, DataSeeder
├─ resources/         i18n properties: EN, KZ, RU
└─ test/             52 tests, custom test framework
```

3.2

3.2.1 Users

```

public abstract class User {
    private final Username username;
    private String passwordHash;
    private final PersonName name;
    private final Gender gender;
    private final LocalDate dateOfBirth;
    private Email email;
    private final Faculty faculty;
    private Language language;

    protected User(Username username, String passwordHash, PersonName name, Gender gender,
                   LocalDate dateOfBirth, Email email, Faculty faculty) {
        this.username = Objects.requireNonNull(username);
        this.passwordHash = Objects.requireNonNull(passwordHash);
        this.name = Objects.requireNonNull(name);
        this.gender = gender;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.faculty = faculty;
        this.language = Language.ENGLISH;
    }

    public Username username() { return username; }
    public String passwordHash() { return passwordHash; }
    public PersonName name() { return name; }
    public Gender gender() { return gender; }
    public LocalDate dateOfBirth() { return dateOfBirth; }
    public Email email() { return email; }
    public Faculty faculty() { return faculty; }
    public Language language() { return language; }

    public void changePassword(String newHash) { this.passwordHash = newHash; }
    public void changeEmail(Email newEmail) { this.email = newEmail; }
    public void changeLanguage(Language lang) { this.language = lang; }

    public boolean matchesPassword(String hash) { return passwordHash.equals(hash); }

    @Override public boolean equals(Object o) {
        return o instanceof User u && username.equals(u.username);
    }
    @Override public int hashCode() { return username.hashCode(); }
    @Override public String toString() { return username + " (" + name.full() + ") [" + getClass().getSimpleName() + "]; }
}

```

User is the root of the user hierarchy. It stores universal attributes — username, passwordHash, name, gender, dateOfBirth, email, faculty, language — and provides general capabilities: change password, change email, switch language, send/view messages, view news, and interact with the research cabinet.

3.2.2 Student

Student extends User and implements ResearcherCapable and BookBorrowerCapable. It carries degreeType (BACHELOR, MASTER, DOCTORATE), studyYear, availableCredits, and failCount (maximum 3 before expulsion). The student can register for disciplines, drop courses, view their

transcript and GPA, borrow library books, submit requests, and join student organizations.

```
public class Student extends User implements BookBorrowerCapable, ResearcherCapable {
    public static final int MAX_FAILS = 3;

    private final DegreeType degreeType;
    private final int studyYear;
    private Credits availableCredits = Credits.SEMESTER_LIMIT;
    private int failCount;

    private final Set<CourseId> enrolled = new LinkedHashSet<>();
    private final Set<CourseId> completed = new LinkedHashSet<>();

    private ResearcherProfile researcherProfile;

    public Student(Username username, String passwordHash, PersonName name, Gender gender,
                  LocalDate dob, Email email, Faculty faculty,
                  DegreeType degreeType, int studyYear) {
        super(username, passwordHash, name, gender, dob, email, faculty);
        this.degreeType = degreeType;
        this.studyYear = studyYear;
    }
}
```

3.2.3 Teacher

```
public class Teacher extends Employee implements BookBorrowerCapable, ResearcherCapable {
    private final String degree;
    private TeacherPosition position;
    private final Set<CourseId> taughtCourses = new LinkedHashSet<>();
    private final List<Integer> ratings = new ArrayList<>();
    private ResearcherProfile researcherProfile;
}
```

Teacher extends Employee (which extends User). It holds salary, hireDate, employeeId, qualification, and position (PROFESSOR, SENIOR_LECTOR, LECTOR, ASSISTANT). Core methods: putMarks(), viewEnrolledStudents(), viewJournal(). Also implements ResearcherCapable and BookBorrowerCapable.

3.2.4 Employee

Employee is an abstract class that extends User. It represents the base class for all university staff members. It stores employee-related information: salary, hireDate, and insuranceNumber.

This class is inherited by Teacher, Manager, Librarian, TechSupport, and EmployeeResearcher.

```
public abstract class Employee extends User {  
    private final Money salary;  
    private final LocalDate hireDate;  
    private final String insuranceNumber;  
}
```

3.2.5 Dean

Dean extends Teacher. In the current implementation, a dean is created as a teacher with the fixed position PROFESSOR.

The dean inherits teacher and researcher functionality. Additionally, the dean can view incoming academic requests and approve or reject them.

3.2.6

CreateManager is an application use case responsible for creating a new manager account. It depends on UserRepository, PasswordHasher, and Logger. The use case hashes the password, creates a Manager domain object, saves it through the repository, and records the action in the system log. This keeps the domain class Manager focused only on storing manager

data, while account creation logic remains in the application layer.

```
17 public final class CreateManager {
18     private final UserRepository users;
19     private final PasswordHasher hasher;
20     private final Logger logger;
21
22     public CreateManager(UserRepository users, PasswordHasher hasher, Logger logger) {
23         this.users = users;
24         this.hasher = hasher;
25         this.logger = logger;
26     }
27
28     public Manager execute(Username actor, String first, String last, String username, String password,
29                           Gender gender, LocalDate dob, String email, Faculty faculty,
30                           double salary, LocalDate hireDate, String insurance, ManagerPosition position) {
31         Manager m = new Manager(new Username(username), hasher.hash(password),
32                                 new PersonName(first, last), gender, dob, new Email(email), faculty,
33                                 new Money(salary), hireDate, insurance, position);
34         users.save(m);
35         logger.log(actor, "Created manager: " + username);
36         return m;
37     }
38 }
```

```
> public final class Manager extends Employee {
    private final ManagerPosition position;
>
    public Manager(Username username, String passwordHash, PersonName name, Gender gender, ...
    public ManagerPosition position() { return position; }
}
```

```
public final class HIndexCalculator {
    public HIndex calculate(List<ResearchPaper> papers) {
        List<Integer> sorted = papers.stream()
            .map(ResearchPaper::citations)
            .sorted(Comparator.reverseOrder())
            .toList();
        int h = 0;
        for (int i = 0; i < sorted.size(); i++) {
            if (sorted.get(i) >= i + 1) h = i + 1;
            else break;
        }
        return new HIndex(h);
    }
}
```

3.3 Central Dependency Container (AppContext)

```
public final CreateStudent createStudent;  
public final CreateGradStudent createGradStudent;  
public final CreateTeacher createTeacher;  
public final CreateDean createDean;  
public final CreateManager createManager;  
public final CreateLibrarian createLibrarian;  
public final CreateTechSupport createTechSupport;
```

```
this.createManager = new CreateManager(userRepository, passwordHasher, logger);  
this.createLibrarian = new CreateLibrarian(userRepository, passwordHasher, logger);  
this.createTechSupport = new CreateTechSupport(userRepository, passwordHasher, logger);  
this.createEmpResearcher = new CreateEmpResearcher(userRepository, passwordHasher, logger);  
this.deleteUser = new DeleteUser(userRepository, logger);  
this.rateTeacher = new RateTeacher(userRepository, logger);  
this.createStudent = new CreateStudent(userRepository, passwordHasher, logger);  
this.createGradStudent = new CreateGradStudent(userRepository, passwordHasher, logger);  
this.createTeacher = new CreateTeacher(userRepository, passwordHasher, logger);  
this.createDean = new CreateDean(userRepository, passwordHasher, logger);
```

AppContext acts as a central dependency container. It creates repositories, services, and use case objects, then provides them to controllers and CLI menus. This avoids creating dependencies manually in every class and keeps the application wiring in one place.

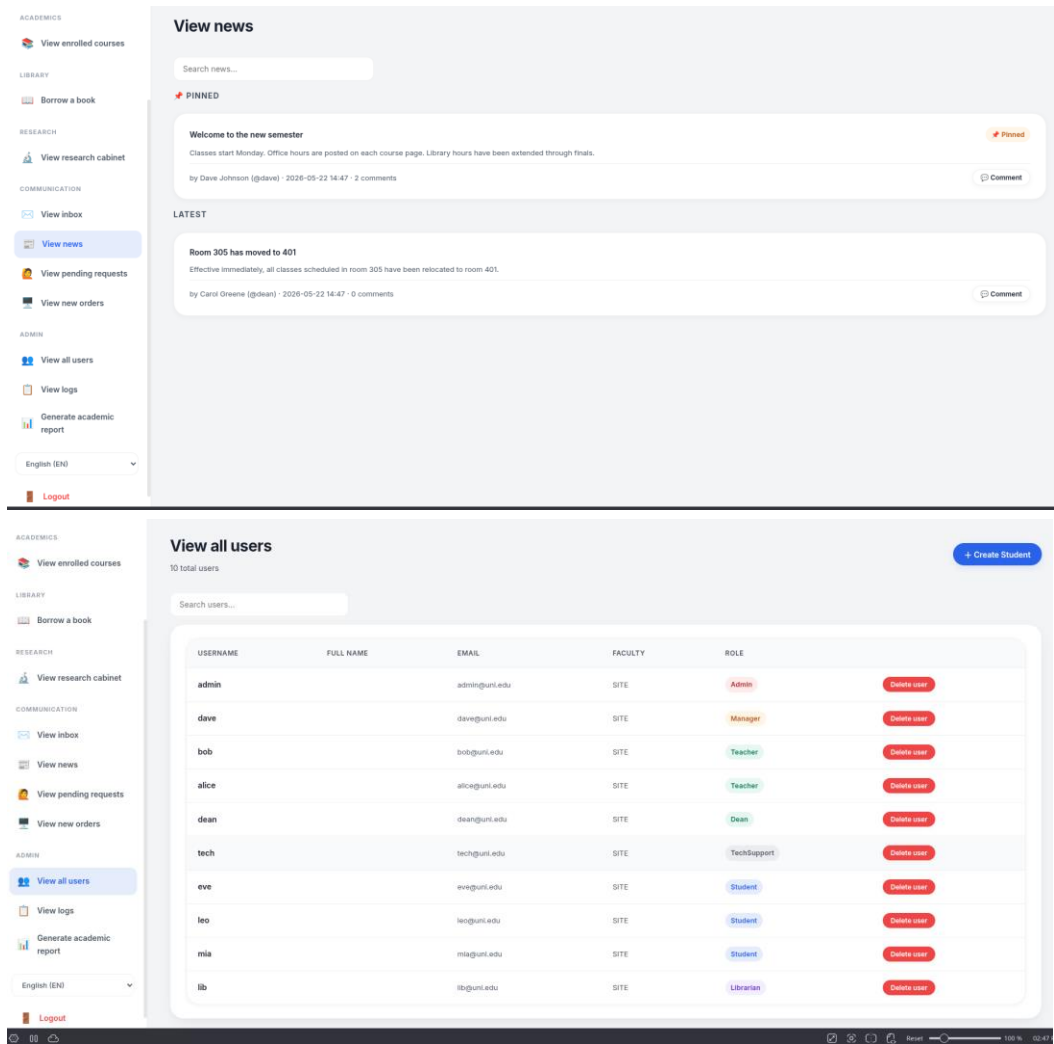
3.4 Frontend

Built with React. 12 pages driven by the REST API.

Page	Purpose
Login	Authentication
Dashboard	Home
Courses	Browse & enroll

Transcript	Student	grades
Gradebook	Teacher	grade entry
Library	Borrow	/ return books
Messages	Inbox	
News	Feed	+ comments
Requests	Help	requests
Orders	IT	support
Research	Papers,	projects, subscriptions
Admin	User	management, logs, reports

Context providers: AuthContext (session state) and I18nContext (language switching between KZ / EN / RU stored in localStorage).



Documentation

OVERVIEWPACKAGECLASSTREEINDEXHELP

SUMMARY: NESTED | FIELD | CONSTR | METHODDETAIL: FIELD | CONSTR | METHOD

Package domain.user

Class Manager

java.lang.Object[®]
domain.user.User
domain.user.Employee
domain.user.Manager

public final class **Manager**
extends Employee

Constructor Summary

Constructors

Constructor	Description
Manager(domain.shared.Username username, String [®] password@hash, domain.shared.PersonName name, domain.enum.Gender gender, LocalDateTime [®] dob, domain.shared.Email email, domain.enums.Faculty faculty, domain.shared.Money salary, LocalDateTime [®] hireDate, String [®] insuranceNumber, domain.enums.ManagerPosition position)	

Method Summary [ⓘ]

All MethodsInstance MethodsConcrete Methods

Modifier and Type	Method	Description
domain.enums.ManagerPosition	position()	

Methods inherited from class domain.user.Employee
hireDate, InsuranceNumber, salary

Methods inherited from class domain.user.User
changeEmail, changeLanguage, changePassword, dateOfBirth, email, equals, faculty, gender, hashCode, language, matchesPassword, name, passwordHash, toString, username

Methods inherited from class java.lang.Object[®]
clone[®], finalize[®], getClass[®], notify[®], notifyAll[®], wait[®], wait[®], wait[®]

Constructor Details

Manager

OVERVIEWPACKAGECLASSTREEINDEXHELP

SUMMARY: NESTED | FIELD | CONSTR | METHODDETAIL: FIELD | CONSTR | METHOD

Package bootstrap

Class AppContext

java.lang.Object[®]
bootstrap.AppContext

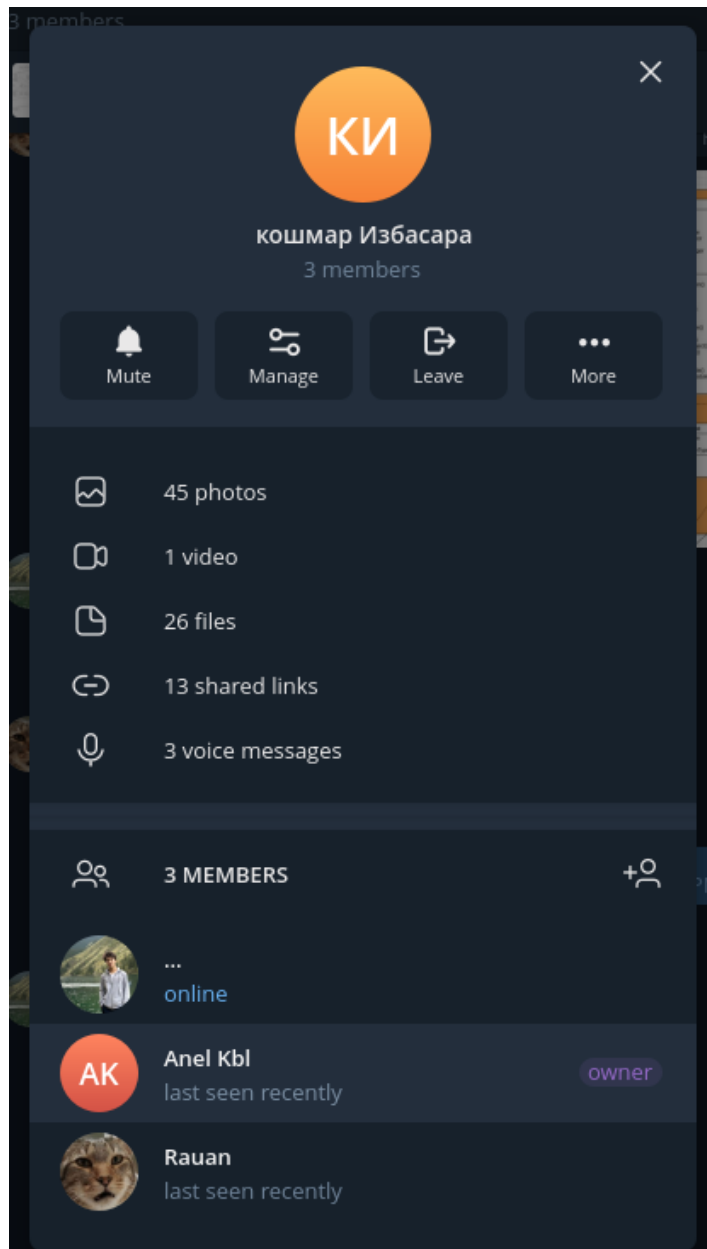
public final class **AppContext**
extends Object[®]

Field Summary

Fields

Modifier and Type	Field	Description
final application.usecase.messaging.AcceptOrder	acceptOrder	
final application.usecase.library.AddBook	addBook	
final AddressLesson	addressLesson	
final AssignTeacher	assignTeacher	
final infrastructure.auth.AuthenticationService	auth	
final application.usecase.user.BecomeResearcher	becomeResearcher	
final presentation.cli.menu.BecomeResearcherAction	becomeResearcherAction	
final domain.shared.IdSequence	bookIds	
final domain.repository.BookRepository	bookRepository	
final application.usecase.library.BorrowBook	borrowBook	
final domain.service.CitationFormatter	citationFormatter	
final application.usecase.messaging.CommentOnNews	commentOnNews	
final application.usecase.user.ComplainAboutStudent	complainAboutStudent	
final application.usecase.messaging.CompleteOrder	completeOrder	
final presentation.cli.Console	console	
final domain.shared.IdSequence	courseIds	
final domain.repository.CourseRepository	courseRepository	
final CreateCourse	createCourse	
final CreateDean	createDean	
final CreateEmpResearcher	createEmpResearcher	
final CreateGradStudent	createGradStudent	
final CreateLibrarian	createLibrarian	
final CreateManager	createManager	
final application.usecase.messaging.CreateITOrder	createOrder	
final application.usecase.organization.CreateOrganization	createOrganization	
final application.usecase.research.CreateResearchProject	createProject	
final CreateStudent	createStudent	
final CreateTeacher	createTeacher	
final CreateTechSupport	createTechSupport	
final infrastructure.persistence.database.Database	database	

Teamworks



Our team coordinated work through a shared Telegram group.

Problems

1. In the design stage, some class relationships were not clear at first, so we had to revise the UML diagram several times.
2. During implementation, it was sometimes difficult to decide whether logic should be placed inside domain classes or separated into use case classes.

3. While working with persistence, we had small issues with saving and loading objects correctly, especially for users with different roles.
4. In the documentation stage, JavaDoc did not generate correctly at first because some package paths were written incorrectly.
5. During testing, some features worked separately, but needed small fixes when connected through menus and controllers.